THE OPEN UNIVERSITY OF SRI LANKA
DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING
BACHELOR OF TECHNOLOGY - LEVEL 6

EEX6535 – COMPILER DESIGN

CONTINUOUS ASSESSMENT TEST #2 - 2019/ 2020 **MODEL ANSWER**

Date: 30 January 2020 2020 **Time: 0900 – 1015 hrs**

[Q1].

VendingMachine → stock STOCK sales SALES
STOCK → STOCK ITEM | ε
ITEM → item price qty
SALES → SALES SALE | ε
SALE → item price

(a) String: ***stock item price qty sales item price***

VendingMachine → stock STOCK sales SALES
→ stock STOCK ITEM sales SALES
→ stock ε item price qty sales SALES
→ stock ε item price qty sales SALES SALE
→ stock ε item price qty sales ε item price [10 Marks]

(b) Each production of the grammar is form of A → BC or A → α and does not contains ε, where A,B,C are non-terminals and α is a terminal [10 Marks]

NOTE: One rule has been missed here --->

(c) Remove all ε production, So **[40 Marks]**
VendingMachine → stock STOCK sales SALES | stock sales SALES | stock sales | stock STOCK sales
STOCK → STOCK ITEM | ITEM
ITEM → item price qty
SALES → SALES SALE | SALE
SALE → item price

NOTE: From now onwards, the above missing rule should be considered. [05 Marks]

Remove all unit productions, So

VendingMachine → stock STOCK sales SALES | stock sales SALES | stock sales
STOCK → STOCK ITEM | item price qty
ITEM → item price qty
SALES → SALES SALE | item price
SALE → item price

[05 Marks]

Now Introduce the intermediate non terminals

VendingMachine → NT1 STOCK NT2 SALES | NT1 NT2 SALES | NT1 NT2
NT1 → stock
NT2 → sales
STOCK → STOCK ITEM | NT3 NT4 NT5
NT3 → item
NT4 → price
NT5 → qty
ITEM → NT3 NT4 NT5
SALES → SALES SALE | NT3 NT4
SALE → NT3 NT4

[10 Marks]

Then the CNF is as follows

VendingMachine → NT6 NT7 | NT1 NT7| NT1 NT2
NT6 → NT1 STOCK
NT7 → NT2 SALES
NT1 → stock
NT2 → sales
STOCK → STOCK ITEM | NT3 NT8
NT8 → NT4 NT5
NT3 → item
NT4 → price
NT5 → qty
ITEM → NT3 NT8
SALES → SALES SALE | NT3 NT4
SALE → NT3 NT4

[20 Marks]

(d) String: ***stock item price qty sales item price***

VendingMachine → NT6 NT7
→ NT1 STOCK NT2 SALES
→ stock STOCK NT2 SALES
→ stock NT3 NT8 NT2 SALES
→ stock item NT8 NT2 SALES
→ stock item NT4 NT5 NT2 SALES
→ stock item price NT5 NT2 SALES
→ stock item price qty NT2 SALES
→ stock item price qty sales SALES
→ stock item price qty sales NT3 NT4
→ stock item price qty sales item NT4

→ stock item price qty sales item price

[10 Marks]

[Q2].

(a) Briefly explain the LR parsing by giving examples. **[10 Marks]**

LR parser is the bottom-up parser which generates the parse tree for the given string by using unambiguous grammar. It follow reverse of right most derivation.

(b) Compare and contrast the top down vs. bottom up parsers. **[20 Marks]**

	TOP DOWN PARSING	BOTTOM UP PARSING
1.	It is a parsing strategy that first looks at the highest level of the parse tree and works down the parse tree by using the rules of grammar.	It is a parsing strategy that first looks at the lowest level of the parse tree and works up the parse tree by using the rules of grammar.
2.	Top-down parsing attempts to find the left most derivations for an input string.	Bottom-up parsing can be defined as an attempts to reduce the input string to start symbol of a grammar.
3.	In this parsing technique we start parsing from top (start symbol of parse tree) to down (the leaf node of parse tree) in top-down manner.	In this parsing technique we start parsing from bottom (leaf node of parse tree) to up (the start symbol of parse tree) in bottom-up manner.
4.	This parsing technique uses Left Most Derivation.	This parsing technique uses Right Most Derivation.
5.	It's main decision is to select what production rule to use in order to construct the string.	It's main decision is to select when to use a production rule to reduce the string to get the starting symbol.

LL PARSER	**LR PARSER**
First L of LL is for left to right and second L is for leftmost derivation.	L of LR is for left to right and R is for rightmost derivation.
It follows the left most derivation.	It follows reverse of right most derivation.
Using LL parser parser tree is constructed in top down manner.	Parser tree is constructed in bottom up manner.
In LL parser, non-terminals are expanded.	In LR parser, terminals are compressed.
Starts with the start symbol(S).	Ends with start symbol(S).

Ends when stack used becomes empty.	Starts with an empty stack.
Pre-order traversal of the parse tree.	Post-order traversal of the parser tree.
Terminal is read after popping out of stack.	Terminal is read before pushing into the stack.
It may use backtracking or dynamic programming.	It uses dynamic programming.
LL is easier to write.	LR is difficult to write.
Example: LL(0), LL(1)	**Example:** LR(0), SLR(1), LALR(1), CLR(1)